NAME:	ID:
NAME:	ID:
NAME:	ID:
NAME:	ID:
NAME:	ID:
DATE:	

Total: _____ / 34

Inertial Measurement Unit (IMU) x TinyML Lab

2103-302 Engineering Measurement

Department of Mechanical Engineering

Chulalongkorn University

Spring 2025

Nattapol Damrongplasit

Lab Contents

i. Lab Objectives
ii. Pre-lab
iii. Lab Sections
iv. Lab Report Submissions

Lab Objectives

In this lab, you will experiment with various types of sensors that can be used to sense motion, vibration, and orientation. You will be using an Arduino Nano 33 BLE Sense board, which has an on-board IMU (Inertial Measurement Unit). You will observe how the accelerometer, gyroscope, and magnetometer acquire physical input data from the environment and output them in a digital form. Furthermore, by leveraging the power of machine learning, we will try to convert physical measurement data into meaningful insight. We will build a real-time gesture recognition device that is able to run machine learning model locally on our microcontroller.

Pre-lab Section

The Nano 33 BLE Sense is packed with built-in sensors such as proximity sensor, colorimeter, gesture, microphone, motion sensor, and temperature/humidy/pressure sensors. These sensors are integrated onto the PCB board that also houses the ARM Cortex-M4 microcontroller and a Bluetooth Low Energy (BLE) module. The best part is that there are no extra wiring or communcation setups required to get started with these sensors. All of the sensors can be accessed by just downloading the corresponding libraries and calling the appropriate functions.

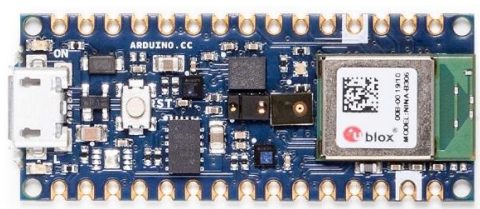


Fig.1 Top-view of the Arduino Nano33 BLE Sense Board

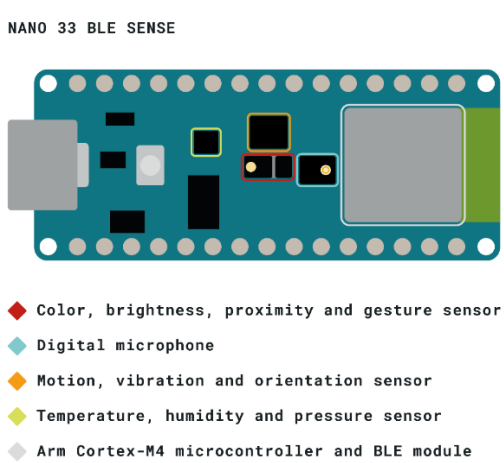


Fig.2 Schematic view of the on-board sensors and ARM microcontroller

IMU module

The IMU (Inertial Measurement Unit) that comes built-in to the Arduino Nano BLE sense board is the LSM9DS1 from STmicroelectronics. It is a system-in-package chip based on MEMS technology that is capable of 9-axis sensing: 3-axis linear acceleration sensor (**accelerometer**), 3-axis angular velocity sensor (**gyroscope**), and 3-axis magnetic sensor (**magnetometer**). Together, these sensors enable us to detect motion, vibration, and orientation.

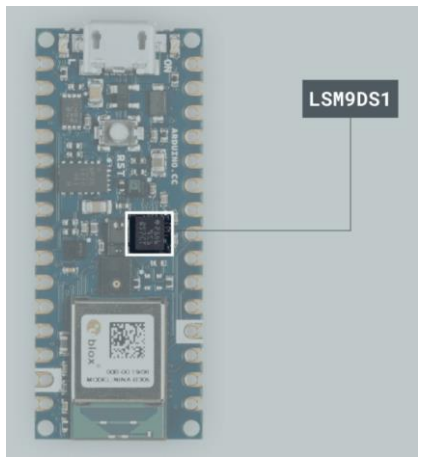


Fig. 3 IMU unit LSM9DS1 by STmicroelectronics surface-mounted onto the board

You can find out more information about the IMU by referring to the manufacturing datasheet. The directions of the sensing axes can be deduced from the diagram in the document. Notice the little white dot painted on the chip packaging, which helps to determine the direction of the axis for the different sensors.

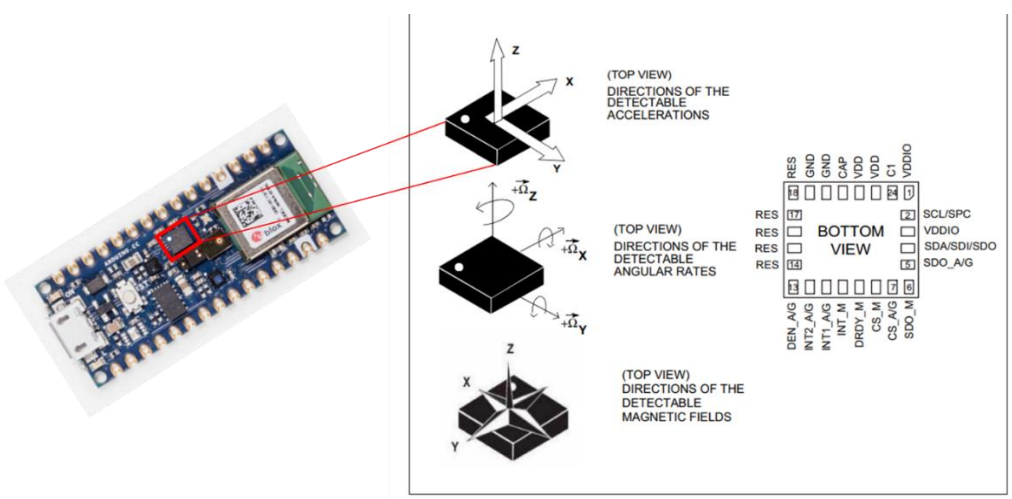


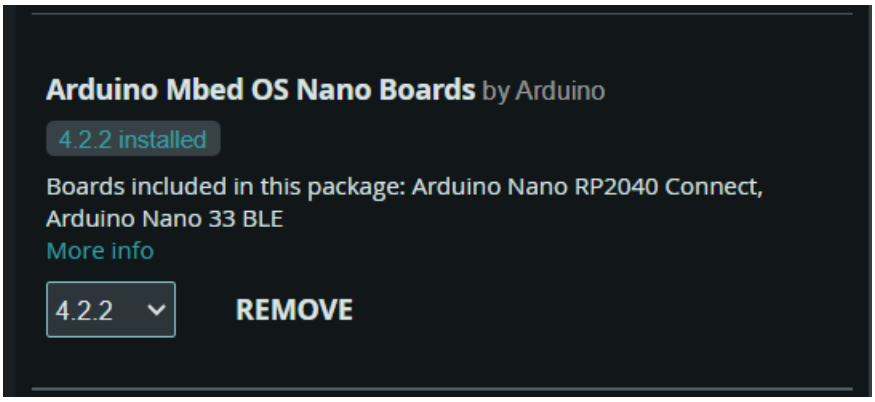
Fig. 4 IMU axis orientation

To use the sensors on the IMU, you will need to install the **LSM9DS1** library. The library takes care of all the initializations, as well as providing an easy-to-use API to obtain sensory data. The range and sampling rate for each sensor is initialized as follows:

- **Accelerometer** range is set at [-4, +4]g -/+0.122 mg. Output data rate is 104 Hz.
- **Gyroscope** range is set at [-2000, +2000] dps (degrees per second) +/-70 mdps. Output data rate is 104 Hz.
- **Magnetometer** range is set at [-400, +400] uT +/-0.014 uT. Output data rate is 20 Hz.

Board Setup

You can download the appropriate driver for Nano 33 BLE Sense board by going to your **Boards Manager** and search for '**Arduino Mbed OS Nano Boards'**. Select install.



Libraries

There are 2 libraries that you need to have installed through the **Library Manager**:

a) **Arduino_LSM9DS1** - This library is for controlling your IMU sensor that is connected on-board to your Nano 33 BLE Sense board

b) **Harvard_TinyMLx** – This library allows you to run TensorFlow machine learning model right on the microcontroller

Capturing IMU Data

Once the LSM9DS1 library is installed, you can use the library by including it at the top of your sketch:

```
#include <Arduino_LSM9DS1.h>
```

To initialize the IMU, the following command can be included in the void setup() section:

```
if (!IMU.begin()) {
      Serial.println("Failed to initialize IMU!");
      while (1);
 }
```

To read the linear acceleration value from the 3-axis, we can use the following commands:

```
float x, y, z;
if (IMU.accelerationAvailable()) {
      IMU.readAcceleration(x, y, z);
 }
```

A gyroscope data can be read in a similar manner as follows:

```
float x, y, z;
if (IMU.gyroscopeAvailable()) {
      IMU.readGyroscope(x, y, z);
 }
```

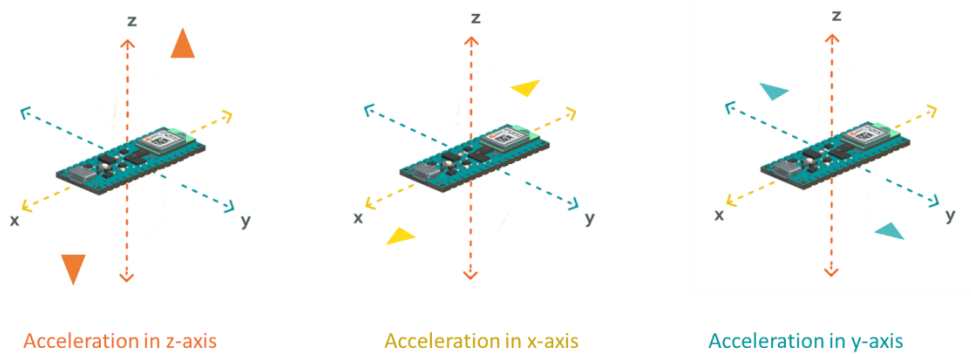
Magnetometer data can be read as follows:

```
float x, y, z;
if (IMU.magneticFieldAvailable()) {
    IMU.readMagneticField(x, y, z);
```

Lab Section

i. Accelerometer

Let's first try to read the acceleration data from the accelerometer. You can run the simple accelerometer sketch using **SimpleAccelerometer.ino** . You can use serial monitor to read the value for the linear accelerations in x, y, and z axis. Note that the value is expressed in the unit of G's (9.81 m/s^2).



Placing the board 'face up' on the table, what values do you get for the X, Y, Z accelerations?

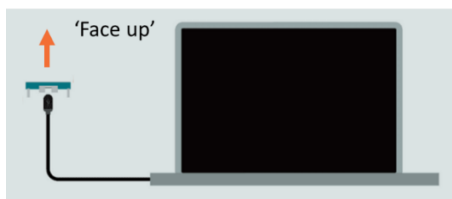


Fig. 5 'Face up' direction

Score ____ / 3

Acceleration along X-axis = _____ [G's]

Acceleration along Y-axis = _____ [G's]

Acceleration along Z-axis = _____ [G's]

Hopefully, you can see that even without exerting any external force on the board, the accelerometer is able to detect the force due to gravity!

Using the serial plotter take a screenshot of acceleration data and paste it below:

Score ____ / 2

Now, let's pick up the board from the table and try to rotate it around at various angles. Observe how the values of x, y, z change according to your movement and rotation. Based on this observation, we could use an accelerometer as a sensor to measure the tilt angle of our device, similar to how a phone is able to detect whether it should display an app in landscape or portrait mode.

A simple tilt angle measurement can be implemented according to **TiltAngle.ino**. The sketch uses the acceleration data in the x and y axis to detect tilting up/down and right/left, respectively (assuming that when the board 'faces up' and the USB input port is on the side hat is closer to you, it has zero tilt angle). A simple mapping function is used to map the raw acceleration value to the amount of tilt in degrees which ranges from 0° to 90°. Try it out and take a screenshot of your serial monitor result below:

Score ____ / 2

Can our accelerometer detect other types of motion? Let's try to move the board in the right, left, front, and back directions according to the suggested diagram below.

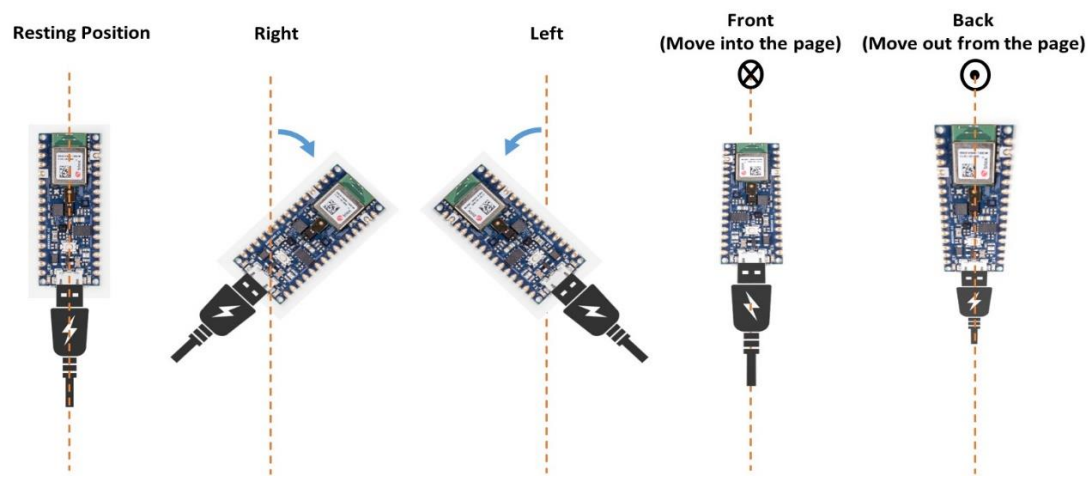


Fig. 6a Various motion gestures: Right, Left, Front, Back

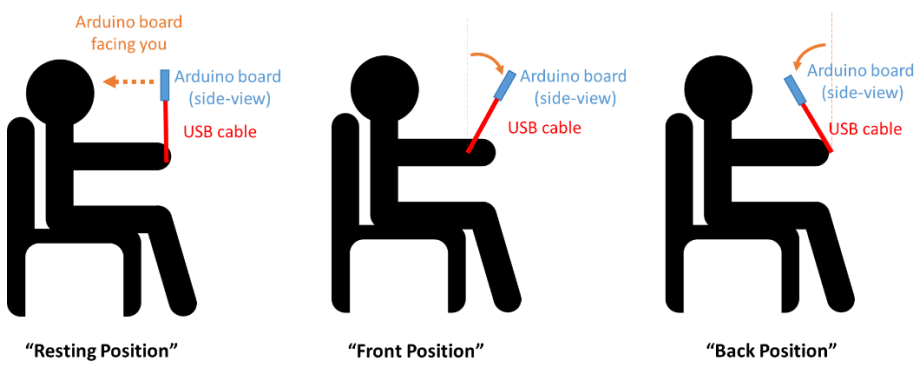


Fig. 6b Various motion gestures (side-view): Resting, Front, Back

Make sure you reload the **SimpleAccelerometer.ino** script (and **NOT the TiltAngle.ino**) since you want to observe the raw acceleration data in the different axes. Please screenshot the serial plotter that capture the motion corresponding to Right, Left, Front, and Back. Be sure to annotate your graph so we know which part of the data corresponds to which motion.

Score ____ / 4

Right Motion:

Left Motion:

Front Motion:

Back Motion:

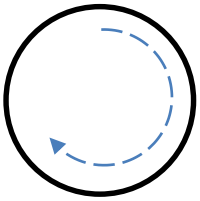
If you were to write a *rule-based* algorithm to detect which of the 4 motions have just occurred, how would you go about doing it? Please describe your idea with a pseudo code or governing equations. You **do not** need to implement a working Arduino sketch for this part.

Score ____ / 2

Now, let's try to do a more complicated gestures, like drawing a circle or another special pattern that you can think of (*Avada Kedavra! Expecto Patronum!)* and capture the accelerometer data below. For your 'special pattern', please sketch how your motion looks like:

Score ____ / 2

Circle:



Special Pattern:

With these gestures, do you think the rule-based algorithm will need to be more complex in order to classify them? Is there a better way than defining more rules? To be continued….

ii. Gyroscope

A gyroscope can detect the angular velocity of an object, or in other words, how fast an object is rotating in degrees per second (dps). The working principle of the device is also fascinating since it relies on Coriolis effect to detect the change in angular velocity. The information generated by a gyroscope can be quite useful on its own. But when you combine this data with an acceleration data, you can achieve an even more accurate assessment of the device's orientation. Recall that an accelerometer can only sense linear acceleration. If your object is subjected to a rotation, the accelerometer will not be able to detect such change. But if you also have a gyroscope on your device, you can certainly detect the rotational change that have just occurred.

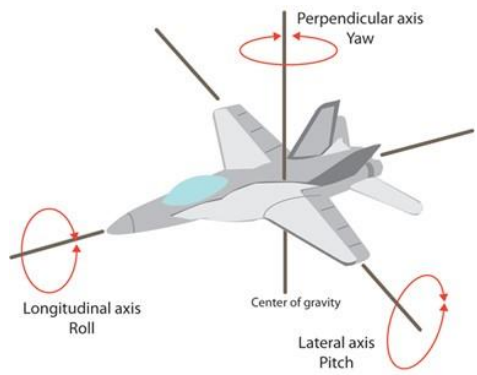


Fig. 7 Rotation around the different axes can be characterized as Roll, Pitch, and Yaw

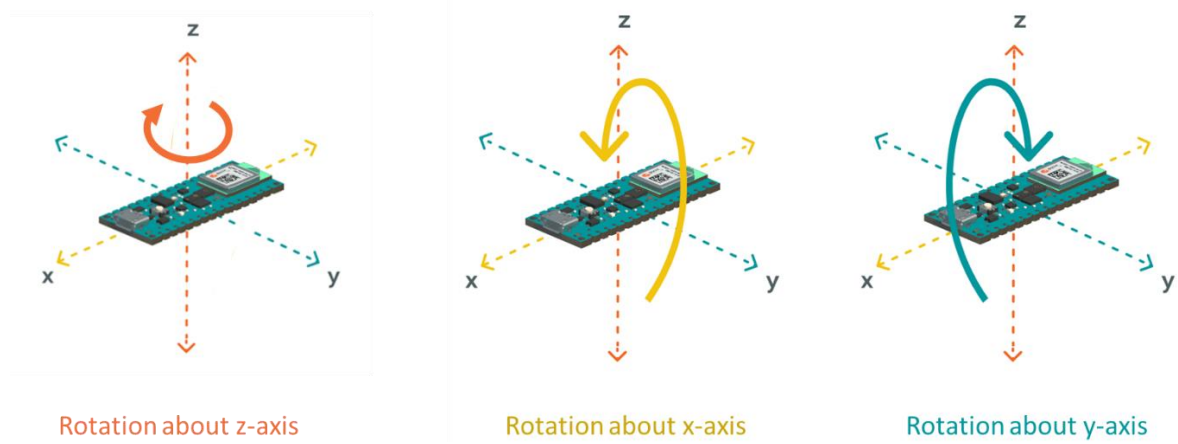


Fig. 8 Rotation around the z, x, y axis with respect to the board position

To test the functionality of a gyroscope, you can upload SimpleGyro.ino file to the board. Try to rotate the board around each of the axes and observe the result on the Serial Plotter. You can try to rotate it in the clockwise (CW) direction for several times (3 – 4 times should be enough) and then follow by consecutive rotations in the counter-clockwise (CCW) direction. Please take

a screenshot of your result and include them below. Be sure to annotate your result so that we can tell which sequences are CW or CCW directions.

Screenshot of angular velocity about the Z-axis:

Score ____ / 1

Screenshot of angular velocity about the X-axis:

Score ____ / 1

Screenshot of angular velocity about the Y-axis:

Score ____ / 1

Based on the result you got, for a given axis rotation, how can you distinguish a CW from a CCW rotations?

Score ____ / 2

Magnetometer

A magnetometer is a device that can measure a magnetic field strength. You can think of it like a digital compass. Again, when used in conjunction with the accelerometer and the gyroscope, we can further improve the accuracy of our device's position and heading. In fact, navigation app on your phone most likely uses a magnetometer to determine the direction that you are facing.

To get started with a magnetometer, you can run **SimpleMagnet.ino** sketch. This code will output the magnetometer data in X, Y, Z axis in the serial monitor, and it will also vary the brightness of its on-board orange LED (located on right side of the USB input) according to the magnetic field strength along the x-axis.

Please take a screenshot of the serial plotter and paste it below. In your plot, try to capture the part where you just let the board sits on the table by itself and when you suddenly bring some electronic components (i.e. cell phone, wireless headphones), electrical wire with current flowing, or a permanent magnet near your Nano sense board. Did you observe any changes? Please be sure to annotate your plot to indicate the different events and their corresponding times.

Score ____ / 2

iii. TinyML

We have seen great advancement in the field of Machine Learning in the past couple of years. Some of the tasks such as visual recognition or speech processing, which appear to be rather daunting for a computer to perform not too long ago, can now be accomplished easily using machine learning – and in some cases the computer can do a better job than us human! The proliferation of such technology will surely open up new possibilities and applications relating to self-driving car, natural language processing, robotic process automation, and data prediction. Nevertheless, the computation involved in training a ML model does not come cheap. Enormous amount of data need to be collected and fed to the model to be trained on a powerful computing cluster. Often, these computers must be equipped with a high-end GPU, TPU, FPGA, or ASIC in order to generate an ever increasingly complex model. Thus, the cost of computation is becoming prohibitively expensive. A large language model (LLM) is estimated to cost about $100 million dollars! That is a lot of dough.

Sitting at the opposite end of the spectrum from these powerful computers and data centers are the low-energy microcontrollers and Internet of Things (IoTs) devices. The growing adoption of these embedded devices is happening at a rapid pace – they are being integrated into consumer electronics, mass transit system, buildings, and other infrastructures. So the question arises: “Can we implement machine learning on these tiny devices, which has a fraction of the computing power, a fraction of a memory storage space, and a fraction of a power budget?”

The answer is a resounding “Yes!”, and we will have the chance to explore it hands-on in this part of the lab. We will implement a ML model into our Nano sense board so that it is capable of recognizing different simple gestures like Right, Left, Front, and Back which we have discussed before. We will use both the accelerometer and gyroscope data as our input to train our deep learning (DL) model. Once the model is trained, we will use TensorFlow Lite to compress our original full DL model into a smaller model without taking too much of a hit in terms of accuracy by using neat little tricks like quantization, pruning, and knowledge-distillation. A smaller size model will be perfect for our microcontroller board to handle given its resource-constraints. So let’s get started!

A) Data capturing

To capture the motion of our gesture, we will record its corresponding acceleration and angular velocity data for the duration of performing a gesture. We can run **IMU_capture.ino** to observe how the script can be used to capture 3-axis accelerometer and 3-axis gyroscope data. The script specifies the number of samples to be captured per event, and it defines the threshold for linear acceleration to determine when significant motion is detected so it knows when to start the recording. Each time that you move the board and the corresponding acceleration

exceeds the threshold value, you should see new data being populated in the serial monitor. The number of lines will be equal the number of samples you have set for each capture. The default is set at 119 samples per capture, and the acceleration threshold is set at 2.5 G's.

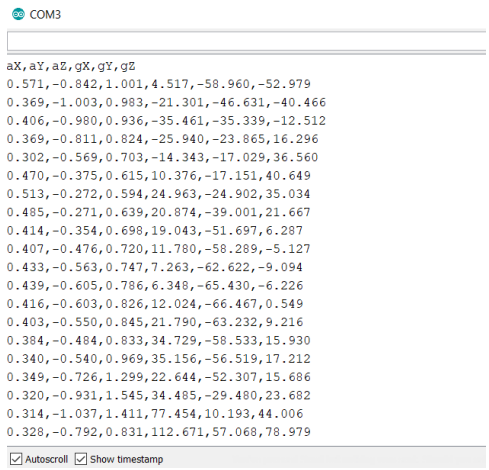


Fig. 9 Serial monitor output after one capture. There are 6 data field (i.e. 3-axis acceleration, 3-axis angular velocity) for the recorded data.

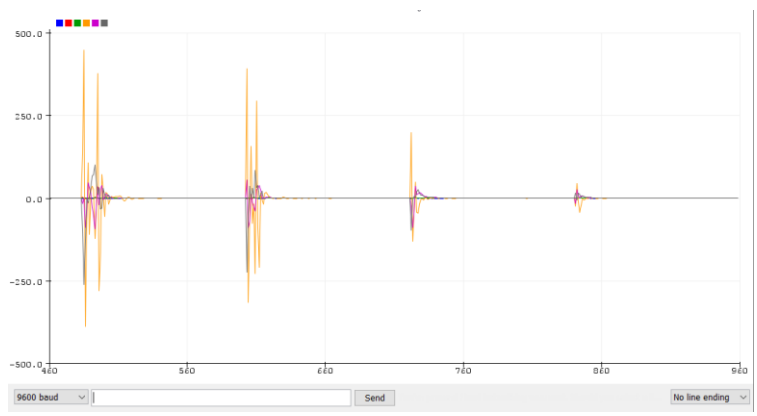


Fig. 10 Serial plotter shows 4 consecutive sets of captured data being recorded.

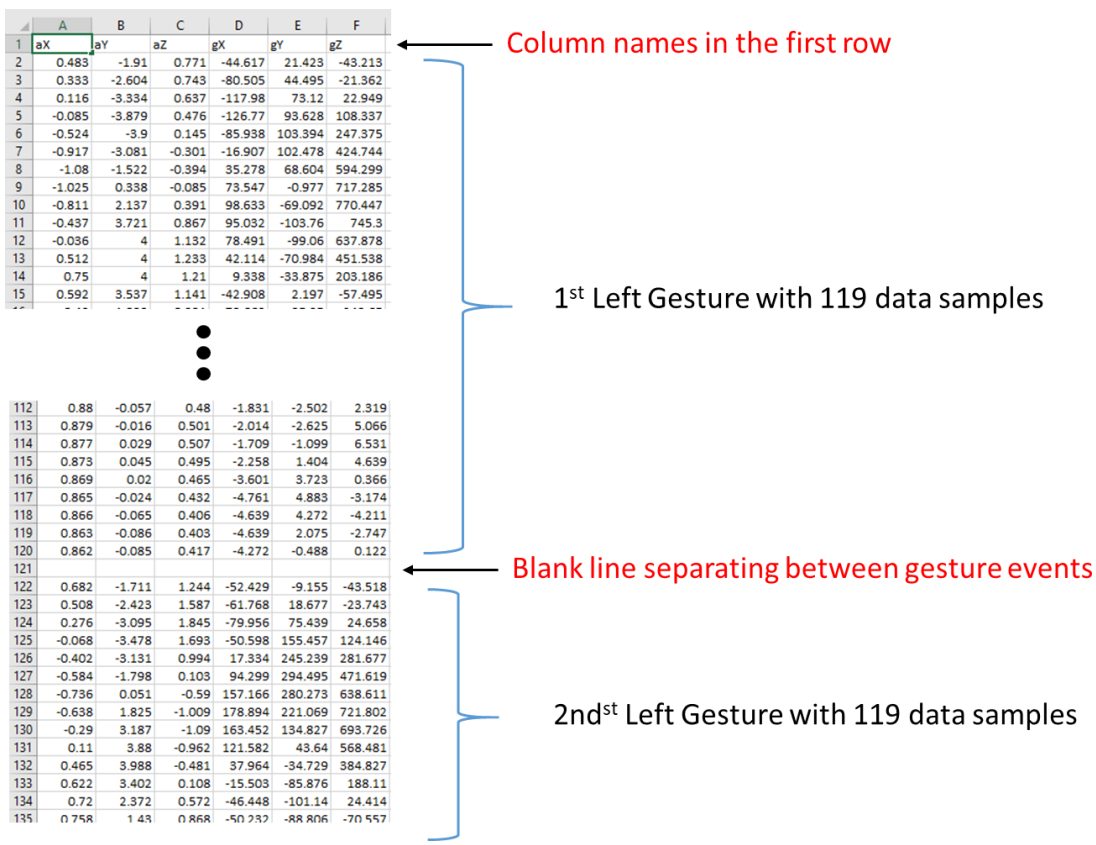
Now try to perform some gestures that you want to classify such as moving the board to the left, right, up, or down. Are there any common patterns in the signal that you might observe when performing the same gesture? Maybe the ML model will be able to pick up the subtle similarities and differences among gestures that might not be as obvious to us.

Now that you have enough fun trying out different gestures, it is time to create a dataset for each gestures to be used in the training of our ML model. **We will try to develop a model that can classify 4 gestures: Left, Right, Circle, and your Special Pattern**. (This special pattern can be anything you want and it doesn't have to be the same special pattern you did previously). For each gesture, please capture **at least 10 sets of data**. Put them in a separate CSV files with

unique names (i.e. Left.csv, Right.csv, Circle.csv, Special.csv). You can copy and paste the data from the serial monitor into your spreadsheet. With Arduino 2.0 IDE, some have reported that they cannot copy over the values from the serial monitor. Alternative solutions are to download an older version of Arduino IDE (1.8xx or pre 2.0) or download a PuTTY terminal to capture the data from the terminal instead.

Also, when capturing the data, be mindful not to inadvertently trigger the data capturing due to any sudden movement of the board. The threshold is set to 2.5G, meaning that if the sensor detects an acceleration greater than this value, it will falsely think that the motion that has just occurred should also be included as part of the sample data and prints it to the terminal.

You CSV data should contain the column header on the first row, and followed by the data. Make sure that there is a blank line between each set of data. This is the default format that you would get if you copy and paste from your serial monitor. As a sanity check, it is a good idea to verify that the number of rows in each of your CSV file are the same. See example below for what the final CSV file should look like:



	A	B	C	D	E	F
1	aX	aY	aZ	gX	gY	gZ
2	0.483	-1.91	0.771	-44.617	21.423	-43.213
3	0.333	-2.604	0.743	-80.505	44.495	-21.362
4	0.116	-3.334	0.637	-117.98	73.12	22.949
5	-0.085	-3.879	0.476	-126.77	93.628	108.337
6	-0.524	-3.9	0.145	-85.938	103.394	247.375
7	-0.917	-3.081	-0.301	-16.907	102.478	424.744
8	-1.08	-1.522	-0.394	35.278	68.604	594.299
9	-1.025	0.338	-0.085	73.547	-0.977	717.285
10	-0.811	2.137	0.391	98.633	-69.092	770.447
11	-0.437	3.721	0.867	95.032	-103.76	745.3
12	-0.036	4	1.132	78.491	-99.06	637.878
13	0.512	4	1.233	42.114	-70.984	451.538
14	0.75	4	1.21	9.338	-33.875	203.186
15	0.592	3.537	1.141	-42.908	2.197	-57.495
⋮						
112	0.88	-0.057	0.48	-1.831	-2.502	2.319
113	0.879	-0.016	0.501	-2.014	-2.625	5.066
114	0.877	0.029	0.507	-1.709	-1.099	6.531
115	0.873	0.045	0.495	-2.258	1.404	4.639
116	0.869	0.02	0.465	-3.601	3.723	0.366
117	0.865	-0.024	0.432	-4.761	4.883	-3.174
118	0.866	-0.065	0.406	-4.639	4.272	-4.211
119	0.863	-0.086	0.403	-4.639	2.075	-2.747
120	0.862	-0.085	0.417	-4.272	-0.488	0.122
121						
122	0.682	-1.711	1.244	-52.429	-9.155	-43.518
123	0.508	-2.423	1.587	-61.768	18.677	-23.743
124	0.276	-3.095	1.845	-79.956	75.439	24.658
125	-0.068	-3.478	1.693	-50.598	155.457	124.146
126	-0.402	-3.131	0.994	17.334	245.239	281.677
127	-0.584	-1.798	0.103	94.299	294.495	471.619
128	-0.736	0.051	-0.59	157.166	280.273	638.611
129	-0.638	1.825	-1.009	178.894	221.069	721.802
130	-0.29	3.187	-1.09	163.452	134.827	693.726
131	0.11	3.88	-0.962	121.582	43.64	568.481
132	0.465	3.988	-0.481	37.964	-34.729	384.827
133	0.622	3.402	0.108	-15.503	-85.876	188.11
134	0.72	2.372	0.572	-46.448	-101.14	24.414
135	0.758	1.43	0.868	-50.232	-88.806	-70.557

Fig. 11 CSV file data format for each gesture

Once you are done collecting the data for each of your gesture, you should have 4 CSV files that are properly named as shown below.

Name	Type
Left	Microsoft Excel Comma Separated Values File
Right	Microsoft Excel Comma Separated Values File
Circle	Microsoft Excel Comma Separated Values File
Special	Microsoft Excel Comma Separated Values File

Fig. 12 Each gesture should have its own CSV file.

B) Model Training with TensorFlow

We will use Google Colab to train the data we have collected to generate our ML model. You will need to have a Google account in order to connect to Google Colab. Please use the link below to open the Python notebook file called GestureRecogntion.ipynb

https://colab.research.google.com/drive/14NugmolR6S42xQQpr70qrYweqKcKpgJc?usp=sharing

You can make a copy of the notebook into your own Google Drive by choosing File > Save a copy in Drive:

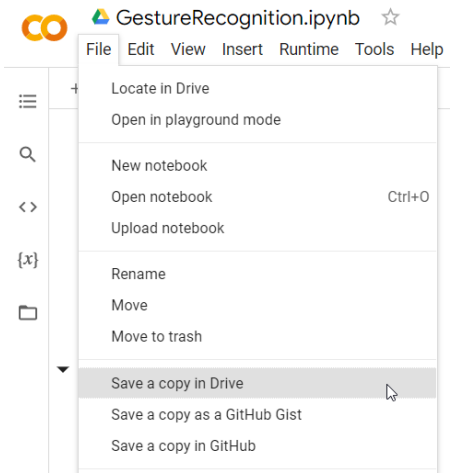


Fig. 13 Saving a copy of the notebook to your Google Drive

The notebook is self-guided so you should be able to execute each section of the code step-by-step.

CSV uploading to Google Colab

When uploading your CSV data, you should click on the File icon to show the current directory, which is set to '/content' by default. Then, you can simply drag-and-drop your CSV files into that folder. Once the files have been copied over, you should see your CSV files appearing in the Google folder directory.

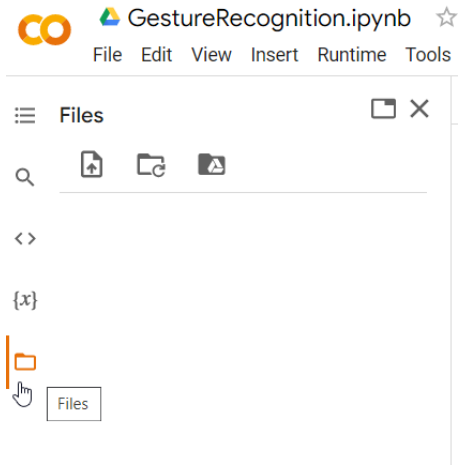


Fig. 14 Click on the File icon on the left vertical tab to show the current directory

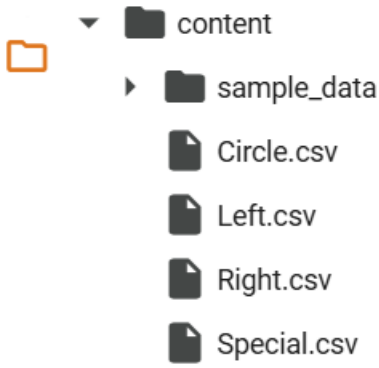


Fig.15 After drag-and-drop your CSV files into the directory, the 4 CSV files should appear

C) Data Exploratory

It is always a good idea to visualize the input data that you are working with. This step is often called data exploratory. We will graph the accelerometer and gyroscope data from our 'left.csv' file to make sure they appear as intended. Compile the python script in the 'Graph Data' section. Please screenshot and post the result below. Of course, you are encouraged to do a visual check on the other CSV files as well.

Acceleration data from left.csv

Score ____ / 2

Gyroscope data from left.csv

Score ____ / 2

D) Verify the model

After you have trained your neural network, let's try to graph the loss curve for your trained model. This plot should contain both the training and validation loss.

Score ____ / 2

E) Convert Trained Model to TensorFlow Lite

Once you are satisfied with the accuracy of your model, the next step is to convert it to a TensorFlow Lite model. After you have run the code conversation script, click on the Refresh button located on the left-side window pane. The file 'gesture_model.tflite' should appear in your default 'content' folder. In the output console of the notebook, you can see the small size of this ML model after the conversion (~ hundreds of kilobytes)

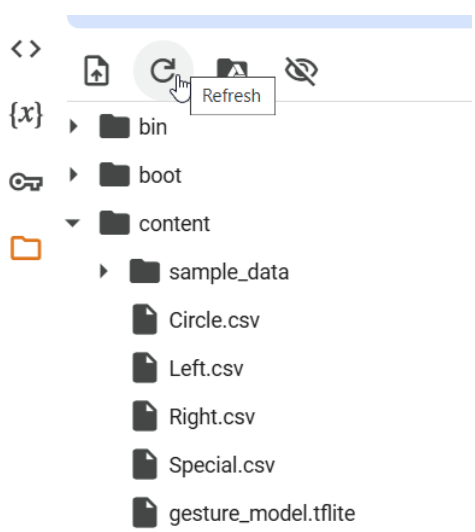


Fig. 16 You might need to refresh the current directory to see the generated .tflite file

F) Encode the model in an Arduino header file

The last step in Google Colab is to transfer the TFlite model into a byte array that Arduino can understand and read from the header file. After you have executed the code, locate the '**model.h'** in the content folder and 'Download' it to your local computer.

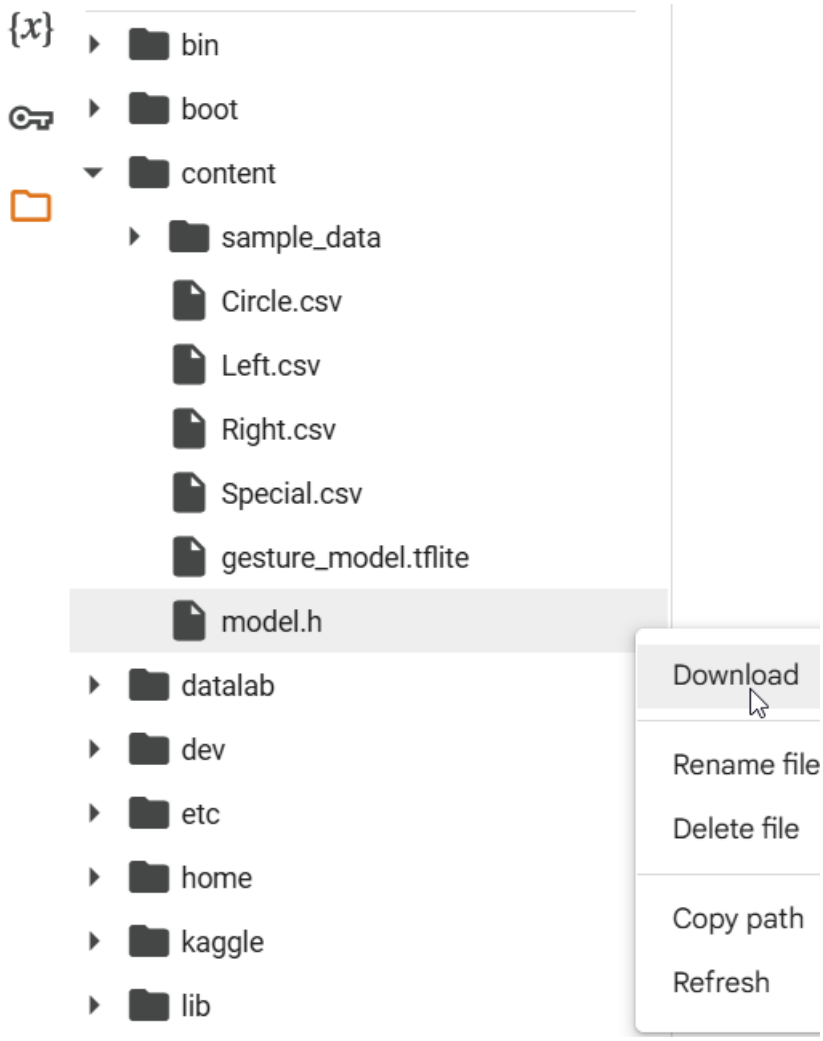


Fig.17 Generate model.h file and download it to your local computer

G) Classifying IMU data

We will be using the following sketch, **IMUclassifier.ino**, to run gesture recognition model right on the Nano Sense board. Since this sketch needs to include the **model.h** file that you have just created, please make sure that you put the header file in the same folder that you are running the sketch, or as part of your computer's default path.

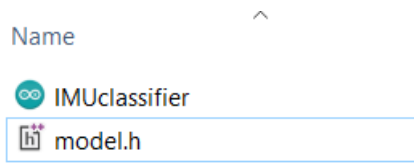


Fig. 18 You can drop model.h in the same folder as your IMUcliassfier.ino sketch

Load the sketch onto the board. The compilation can take a while (up to a minute or two). Once it is completed, open up the Serial Monitor. Now, perform the gesture and see what our TinyML predicts. A confidence level for each gesture is printed, a 0 being low confidence and 1 being a high confidence. A sample output is shown below. Please take a screenshot of your serial monitor result and describe qualitatively how accurate are the predictions.

```
Left: 0.999453
Right: 0.000001
Circle: 0.000000
Special: 0.000546
```

Predicted 'Left' with 99.9% confidence

```
Left: 0.000574
Right: 0.976005
Circle: 0.000000
Special: 0.023421
```

Predicted 'Right' with 97.6% confidence

```
Left: 0.001386
Right: 0.044265
Circle: 0.954349
Special: 0.000000
```

Predicted 'Circle' with 95.4% confidence

```
Left: 0.350372
Right: 0.000833
Circle: 0.000000
Special: 0.648795
```

Predicted 'Special' with 64.8%, but also has a moderate confidence of 35% of predicting 'Left' instead

Fig. 19 Sample of a Serial monitor output after each gesture is performed

Output of IMU classification

Score ____ / 4

Discussion of the performance of your tinyML device (accuracy, latency, responsiveness, etc…). Suggestion on how you can improve the accuracy of the model?

Score ____ / 2

Congratulation! You have just trained your first machine learning model and implemented it right on a microcontroller board that is a size of a bubble gum! What other interesting applications can you think of that might benefit from this technology? The possibilities are endless! But one thing is certain: There is definitely a **BIG** opportunity for this **Tiny**ML devices ☺.

Lab Submissions

Make sure you have completed all the questions in the pre-lab and lab sections. Only one copy per group is required for submission. Please submit your lab report **electronically** through myCourseville.

References

1. Official Arduino Nano 33 BLE Sense online documents (https://docs.arduino.cc/hardware/nano-33-ble-sense)
2. LSM9DS1 datasheet (https://content.arduino.cc/assets/Nano_BLE_Sense_lsm9ds1.pdf)
3. TensorFlowLite (https://www.tensorflow.org/lite)